

An aerial, high-angle photograph of a city street filled with yellow taxis. The taxis are arranged in a grid-like pattern, moving in different directions. The street has white lane markings and arrows. The taxis have various colored signs on their roofs: white, orange, and blue. The overall scene is a busy urban environment.

Taxi Planning

Giovanni Di Nola 1894682

Serena Ragaglia 1941007

Domain

The **goal** is to manage the movement of a taxi in a map of interconnected location, handling traffic, clients' requests and battery level.

Passengers are in different locations and have an active request to reach a destination. The taxi has to reach the location, pick up the passenger, move to passenger's destination and drop them off.

The taxi has limited **battery**, that can be recharged by reaching a recharge station, a specific type of location.

Traffic **lights** and **congestions** increase the cost of crossing specific locations and roads.



PDDL

- **(road ?l1 ?l2 – location)**

Indicates that there is a direct road connection from location `?l1` to location `?l2`. This represents the physical infrastructure allowing the taxi to travel between two locations.

- **(trafficLight ?l1 ?l2 - location)**

Indicates that there is a traffic light controlling the road from location `?l1` to location `?l2`. The taxi must check the light color before passing through.

- **(lightColor ?c - color ?l1 ?l2 - location)**

Specifies the current color `?c` of the traffic light on the road from `?l1` to `?l2`. Used to determine if the taxi can proceed through a traffic light.

- **(taxiAt ?l - location)**

Indicates the current location of the taxi. The taxi can only be at one location at a time.

PDDL

- **(passengerAt ?p - passenger ?l - location)**

Indicates that passenger `?p` is currently at location `?l`. Used to track passenger positions when they are not in the taxi.

- **(requestTo ?p - passenger ?l - location)**

Indicates that passenger `?p` has requested transportation to location `?l`. This represents the passenger's desired destination.

- **(onTaxi ?p - passenger)**

Indicates that passenger `?p` is currently riding in the taxi. Only one passenger can be in the taxi at a time in this domain.

- **(congested ?l - location)**

Indicates that location `?l` is congested (high traffic). Travel costs are increased when entering congested locations.

- **(chargeStation ?l - location)**

Indicates that location `?l` has a charging station where the taxi can recharge its battery.

Actions

Move (?l1 ?l2 - location)	Taxi must be at location ?l1. A road must exist from ?l1 to ?l2. Battery level must be sufficient to cover the distance. If there's a traffic light, it must be green.	Taxi is now at location ?l2 (no longer at ?l1). Battery level decreases by distance × cost-per-distance. If ?l2 is congested: total cost increases by distance × 5. If ?l2 is not congested: total cost increases by distance. If there was a green traffic light: it switches to red.
switchLight (?l1 ?l2 - location)	A traffic light must exist between ?l1 and ?l2. The light must currently be red.	Light color changes from red to green. Total cost increases by 3.
Recharge (?l - location)	Location ?l must have a charging station. Taxi must be at location ?l. Battery level must be less than 100.	Battery level is set to 100. Total cost increases by 10.

Actions

pickUp (?p - passenger ?l - location)	Passenger ?p must be at location ?l. Taxi must be at location ?l. Passenger must have an active request. No other passenger can be in the taxi.	Passenger is no longer at location ?l. Passenger is now on the taxi.
getOff (?p - passenger ?l - location)	Passenger ?p must be on the taxi. Taxi must be at location ?l. Location ?l must be the passenger's requested destination.	Passenger is no longer on the taxi. Passenger is now at location ?l. Passenger's request to location ?l is fulfilled.



Problems

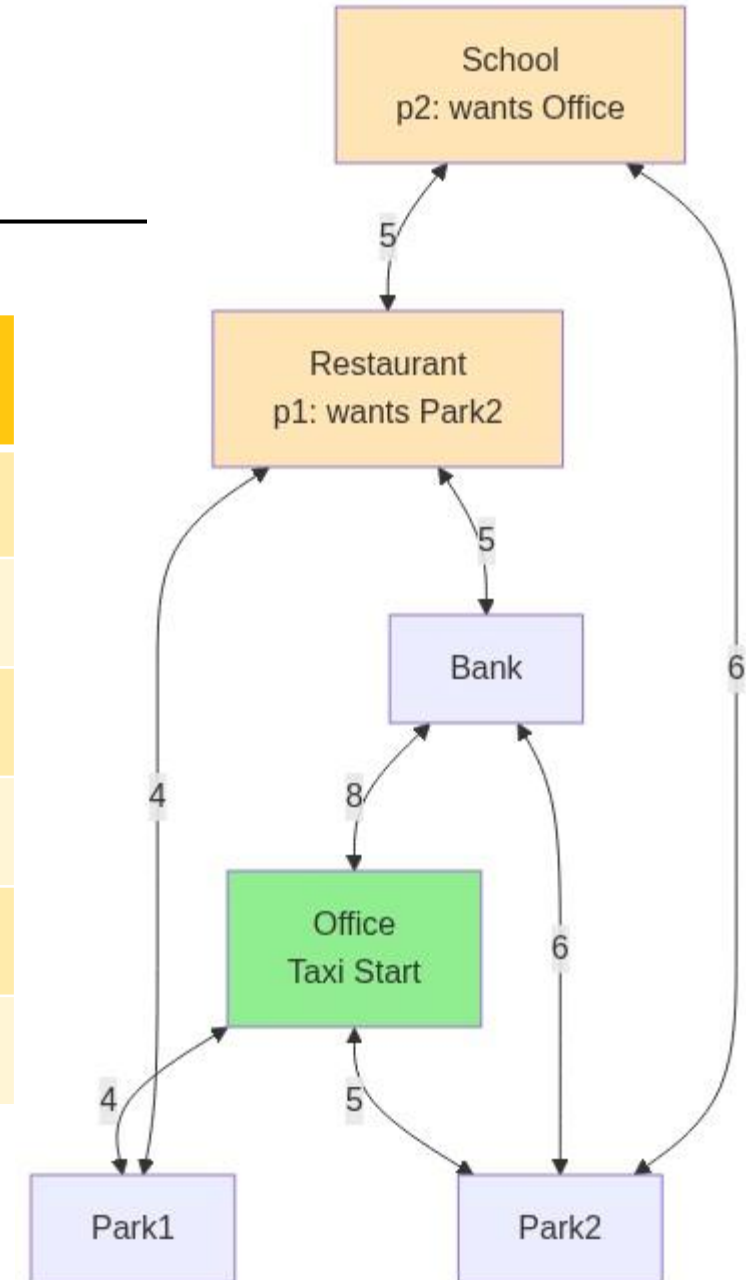
In the problem files the various initial states change based on:

- Number of locations
- Number of requests
- Presence of congested roads
- Presence of traffic lights



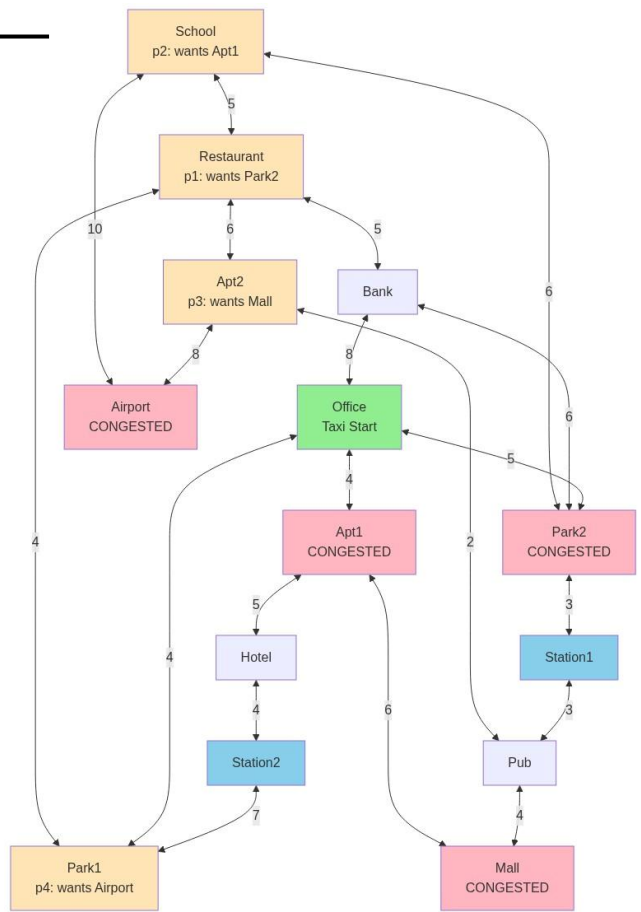
Problem 1

	Plan length	Metric	Planning Time (ms)	Heuristic Time (ms)	Search Time (ms)	Expanded Nodes	States Evaluated
A* h_add	13	288.0	81	35	72	264	322
A* h_max	13	288.0	83	31	73	313	422
Greedy h_add	13	288.0	76	28	71	264	322
Greedy h_max	13	288.0	171	77	156	313	422
WA* h_add	11	216.0	63	15	59	37	91
WA* h_max	11	216.0	74	29	69	143	278



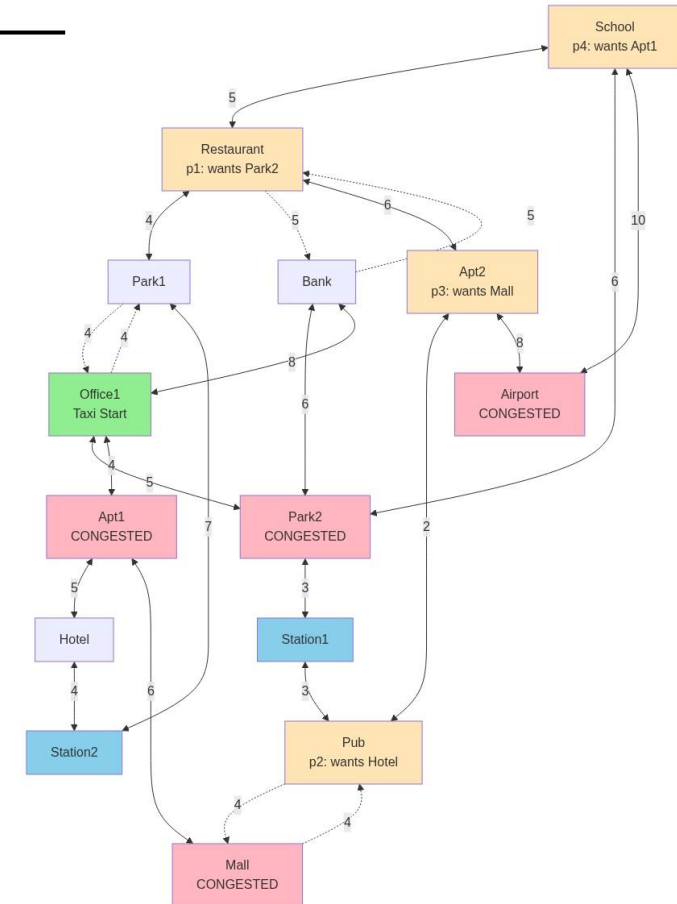
Problem 2

	Plan length	Metric	Planning Time (ms)	Heuristic Time (ms)	Search Time (ms)	Expanded Nodes	States Evaluated
A* h_add	45	888.0	223	151	217	1063	2113
A* h_max	53	966.0	782	646	777	14814	24028
Greedy h_add	45	888.0	195	116	190	1063	2113
Greedy h_max	53	966.0	1031	794	794	1023	14814
WA* h_add	35	614.0	195	122	190	653	1503
WA* h_max	27	526.0	925	606	920	12742	15640

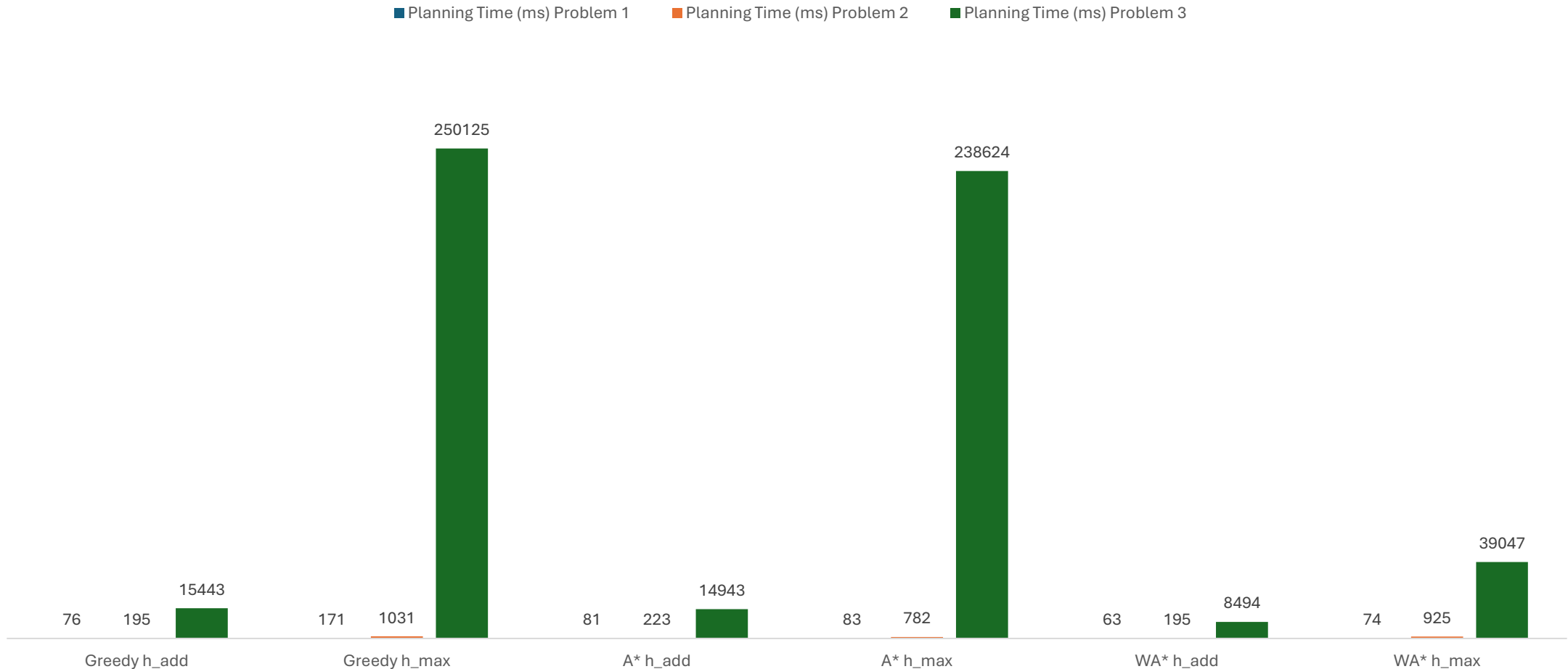


Problem 3

	Plan length	Metric	Planning Time (ms)	Heuristic Time (ms)	Search Time (ms)	Expanded Nodes	States Evaluated
A* h_add	83	1648.0	14943	12987	14939	250202	401887
A* h_max	81	1616	238624	203052	238619	4910931	7168511
Greedy h_add	83	1648.0	15443	13029	15438	250202	401887
Greedy h_max	81	1616.0	250125	211018	250121	4910931	7168511
WA* h_add	39	779.0	8494	7117	8490	118341	211332
WA* h_max	36	674.0	39047	30356	39040	741790	935732



Comparing planning times



Problem 1

WA* h_add

(move office park1)
(move park1 restaurant)
(pickUp p1 restaurant)
(move restaurant school)
(move school park2)
(getOff p1 park2)
(move park2 school)
(pickUp p2 school)
(move school park2)
(move park2 office)
(getOff p2 office)

WA* h_max

(move office park1)
(move park1 restaurant)
(pickUp p1 restaurant)
(move restaurant bank)
(move bank park2)
(getOff p1 park2)
(move park2 school)
(pickUp p2 school)
(move school park2)
(move park2 office)
(getOff p2 office)

Problem 2

WA* h_add

(move office park2)	(move apartment2 pub)	(move office park1)
(move park2 chargeStation1)	(move pub chargeStation1)	(pickUp p4 park1)
(move chargeStation1 pub)	(recharge chargeStation1)	(move park1 office)
(move pub apartment2)	(move chargeStation1 park2)	(move office park2)
(pickUp p3 apartment2)	(getOff p1 park2)	(move park2 chargeStation1)
(move apartment2 pub)	(move park2 school)	(recharge chargeStation1)
(move pub mall)	(pickUp p2 school)	(move chargeStation1 pub)
(getOff p3 mall)	(move school park2)	(move pub apartment2)
(move mall pub)	(move park2 office)	(move apartment2 airport)
(move pub apartment2)	(move office apartment1)	(getOff p4 airport)
(move apartment2 restaurant)	(getOff p2 apartment1)	
(pickUp p1 restaurant)	(move apartment1 office)	
(move restaurant apartment2)		

Problem 2

WA* h_max

(move office park1)
(pickUp p4 park1)
(move park1 restaurant)
(move restaurant apartment2)
(move apartment2 airport)
(getOff p4 airport)
(move airport apartment2)
(pickUp p3 apartment2)
(move apartment2 pub)
(move pub mall)
(getOff p3 mall)
(move mall pub)
(move pub chargeStation1)
(recharge chargeStation1)

(move chargeStation1 pub)
(move pub apartment2)
(move apartment2 restaurant)
(pickUp p1 restaurant)
(move restaurant bank)
(move bank park2)
(getOff p1 park2)
(move park2 school)
(pickUp p2 school)
(move school park2)
(move park2 office)
(move office apartment1)
(getOff p2 apartment1)

Problem 3

WA* h_add

(switchLight office1 park1)

(move office1 park1)

(move park1 restaurant)

(pickUp p1 restaurant)

(move restaurant school)

(move school park2)

(switchLight pub mall)

(getOff p1 park2)

(move park2 chargeStation1)

(move chargeStation1 pub)

(pickUp p2 pub)

(move pub mall)

(move mall apartment1)

(move apartment1 hotel)

(getOff p2 hotel)

(move hotel chargeStation2)

(recharge chargeStation2)

(move chargeStation2 park1)

(move park1 restaurant)

(move restaurant school)

(pickUp p4 school)

(move school park2)

(move park2 office1)

(move office1 apartment1)

(getOff p4 apartment1)

(move apartment1 hotel)

(move hotel chargeStation2)

(recharge chargeStation2)

(move chargeStation2 park1)

(move park1 restaurant)

(move restaurant apartment2)

(pickUp p3 apartment2)

(switchLight park1 office1)

(move apartment2 restaurant)

(move restaurant park1)

(move park1 office1)

(move office1 apartment1)

(move apartment1 mall)

(getOff p3 mall)

Problem 3

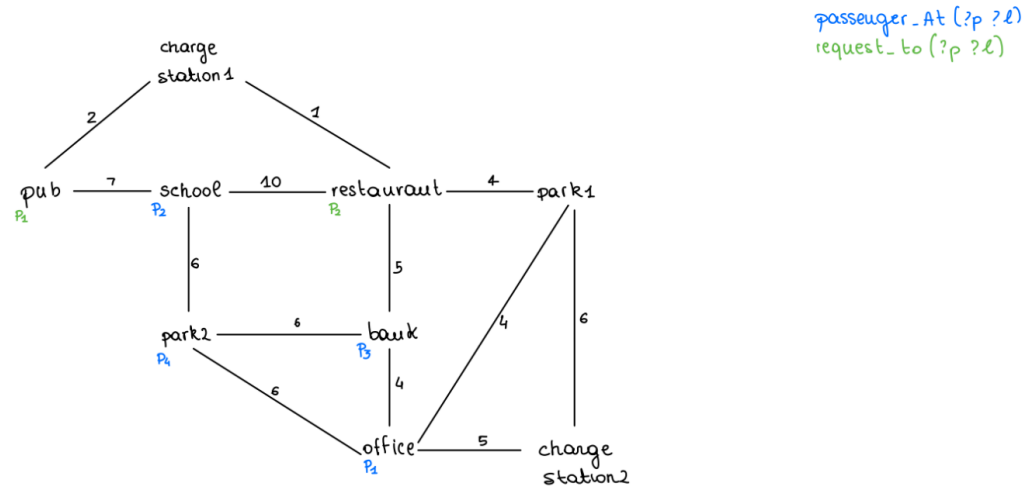
WA* h_max

(switchLight office1 park1)	(move pub mall)	(move park1 restaurant)
(move office1 park1)	(getOff p3 mall)	(pickUp p1 restaurant)
(move park1 restaurant)	(switchLight mall pub)	(move restaurant school)
(move restaurant apartment2)	(move mall pub)	(move school park2)
(pickUp p3 apartment2)	(pickUp p2 pub)	(getOff p1 park2)
(move apartment2 restaurant)	(switchLight pub mall)	(move park2 school)
(move restaurant school)	(move pub mall)	(pickUp p4 school)
(move school park2)	(move mall apartment1)	(move school park2)
(switchLight pub mall)	(move apartment1 hotel)	(move park2 office1)
(move park2 chargeStation1)	(getOff p2 hotel)	(move office1 apartment1)
(recharge chargeStation1)	(move hotel chargeStation2)	(getOff p4 apartment1)
(move chargeStation1 pub)	(recharge chargeStation2)	
	(move chargeStation2 park1)	

IndigoLog

To perform the reasoning task with indigolog, we simplified the domain by using a map similar to the first problem, removing the possibility of traffic lights and congested locations.

We implemented 2 exogenous actions and we studied the behavior of our controllers in response to them.



Relational and functional fluents

- **taxi_at**

rel_fluent(taxi_at(L)) :- location(L).

causes_true(move(_, L2), taxi_at(L2), true).

causes_false(move(L1, _), taxi_at(L1), true).

- **congested**

rel_fluent(congested(L)) :- location(L).

- **on_taxi**

rel_fluent(on_taxi(P)) :- passenger(P).

causes_true(pickUp(P, _), on_taxi(P), true).

causes_false(getOff(P, _), on_taxi(P), true).

- **request_to**

rel_fluent(request_to(P, L)) :- passenger(P), location(L).

causes_false(getOff(P, L), request_to(P, L), true).

- **battery_level**

fun_fluent(battery_level).

causes_val(recharge(_), battery_level, 100, true).

- **passenger_at**

rel_fluent(passenger_at(P, L)) :- passenger(P), location(L).

causes_false(pickUp(P, L), passenger_at(P, L), true).

causes_true(getOff(P, L), passenger_at(P, L), true).

Actions

- `prim_action(move(L1, L2)) :- location(L1), location(L2).`

```
poss(move(L1, L2),  
    and(taxi_at(L1),  
        and(road(L1, L2),  
            (battery_level > (D * C))))))
```

```
:- distance(L1, L2, D), cost_per_distance(C).
```

- `prim_action(recharge(L)) :- location(L).`
`poss(recharge(L),`
 `and(charge_station(L),`
 `and(taxi_at(L),`
 `battery_level < 100)))).`

- `prim_action(pickUp(P, L)) :- passenger(P), location(L).`

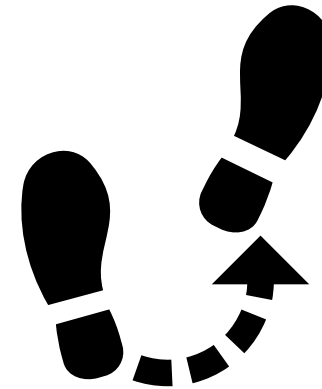
```
poss(pickUp(P, L),  
    and(passenger_at(P, L),  
        and(taxi_at(L),  
            neg(some(j, on_taxi(j)))))).
```

- `prim_action(getOff(P, L)) :- passenger(P), location(L).`
`poss(getOff(P, L),`
 `and(on_taxi(P),`
 `and(taxi_at(L),`
 `request_to(P, L)))).`

Exogenous actions

We added three exogenous actions to test our reactive controller:

- **call**: creates a new request, deleting the old one from the same passenger.
- **move_p**: makes a passenger move from its location to another, like a teleportation. This action has been introduced in order to see how the taxi reacts when a passenger moves.



Legality task

To test the domain, we selected a series of actions as input to the legality task, to check its executability.

The **first** task:

[move(park1, restaurant), move(restaurant, school), pickUp(p2, school), move(school, restaurant), getOff(p2, restaurant)].

proved legal.

The **second** task:

[move(park1, restaurant), move(restaurant, school), pickUp(p2, school), getOff(p2, school)].

proved illegal, due to the lack of a request of p2 to reach school.

Projection task

For the projection task we wanted to check if a series of actions led to a desired state.

Write the condition 'and(f1(), neg(f2()))':

```
|: and(taxi_at(office), neg(request_to(p1, pub)))
```

Write the sequence of actions '[a1(), ..., an()]':

```
|: [move(park1, office), pickUp(p1, office), move(office, park2),  
move(park2, school), move(school, pub), getOff(p1, pub), move(pub,  
school), move(school, park2), move(park2, office)]
```

Condition and(taxi_at(office), neg(request_to(p1, pub))) HOLDS after
executing

```
[move(park1, office), pickUp(p1, office), move(office, park2), move(park2, sch  
ool), move(school, pub), getOff(p1, pub), move(pub, school), move(school, park2  
) , move(park2, office)] true.
```


Controllers

We decided to create different types of controllers.

The **wander** controller performs random actions and has to end up at the restaurant location.

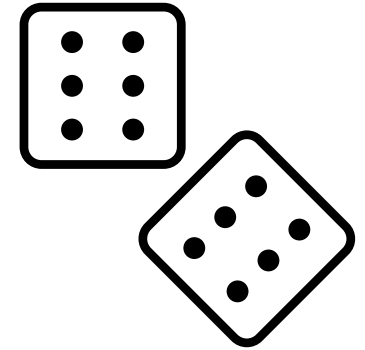
The **dumb** controller also performs random actions, until it completes all passenger's requests.

The **basic-battery** controller has the goal to satisfy all requests, but has to keep an eye on the taxi's battery that can go below a certain level and when it does, it needs to be charged.

The **reactive** controller has to respond to exogenous actions.



Controllers

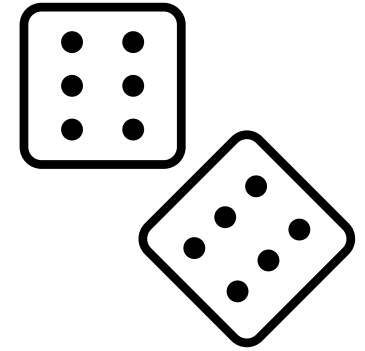


```
proc (control (wander) ,  
  search (wander) ) .  
proc (wander, [  
  star (random_action) ,  
  ? (taxi_at (pub))  
) ) .
```

Execution of control(wander)

```
move (school, pub) , move (restaurant, school) , move (park1, restaurant)
```

Controllers



```
proc(control(dumb), search(dumb)) .  
proc(dumb, [  
    star(random_action),  
    ?(final_condition)  
]) .
```

Execution of control(dumb)

```
getOff(p1,bank),move(office,bank),pickUp(p1,office),  
move(park1,office),move(restaurant,park1),move(park1,  
,restaurant),move(restaurant,park1),getOff(p2,restau  
rant),move(school,restaurant),pickUp(p2,school),move  
(restaurant,school),move(park1,restaurant)
```

Controllers



```
proc (control (basic-battery), search (basic-battery)) .  
proc (basic-battery,  
    while (neg (final_condition),  
        if (battery_level =< 70,  
            go_to_recharge,  
            serve_some_passenger))) .
```

Execution of control(basic-battery)

```
getOff (p2, restaurant), move (school, restaurant), pickUp (p2, school),  
move (pub, school), move (charge_station1, pub), recharge (charge_stati  
on1), move (restaurant, charge_station1), move (bank, restaurant), getO  
ff (p1, bank), move (office, bank), pickUp (p1, office), move (park1, offic  
e), move (restaurant, park1), move (school, restaurant), move (restauran  
t, school), move (park1, restaurant)
```

Controllers



```
proc(control(reactive), [  
  prioritized_interrupts([  
    interrupt(neg(final_condition), [  
      unset(has_changed),  
      gexec(neg(has_changed), search(basic-battery))]),  
    interrupt(true, say("Waiting...")),  
    interrupt(true, ?(wait_exog_action))  
  ])  
]).
```

This controller handles the changes inside the world that can be caused by exogenous actions like the movement or the creation of a new request.

Controllers



Execution of control(reactive)

```
getOff(p3, restaurant), move(bank, restaurant), move(park2, bank), pickUp(p3, park2),  
move(school, park2), move(restaurant, school), move(school, restaurant), move(pub, sc  
hool), move(charge_station1, pub), recharge(charge_station1), move(restaurant, char  
ge_station1), move(park1, restaurant), move(restaurant, park1), move(school, restaur  
ant), move(restaurant, school), call(p3, restaurant), move_p(p3, park2), getOff(p2, re  
staurant), move(school, restaurant), pickUp(p2, school), move(pub, school), move(char  
ge_station1, pub), recharge(charge_station1), move(restaurant, charge_station1), mo  
ve(bank, restaurant), getOff(p1, bank), move(park2, bank), move(school, park2), move(r  
estaurant, school), move(school, restaurant), move(pub, school), pickUp(p1, pub), move  
(charge_station1, pub), recharge(charge_station1), move(pub, charge_station1), call  
(p1, bank), getOff(p1, pub), move(charge_station1, pub), move(restaurant, charge_stat  
ion1), move(bank, restaurant), move(office, bank), pickUp(p1, office), move(bank, offi  
ce), move(park2, bank), move(school, park2), move(restaurant, school), move(park1, res  
taurant), unset(has_changed)
```

Thank you for the attention

https://github.com/serenaragaglia/taxi_planning.git